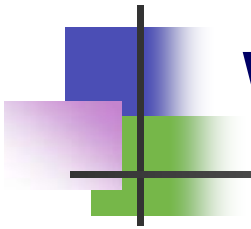
Sorting Algorithms

Jim D. Pham

3/29/2017



Why sorting?

- Sorting is any process of rearranging the data in a more meaningful order.

Understanding Lists

- The array stores a sequence of values that are all of the same type.
  - Array of any data type

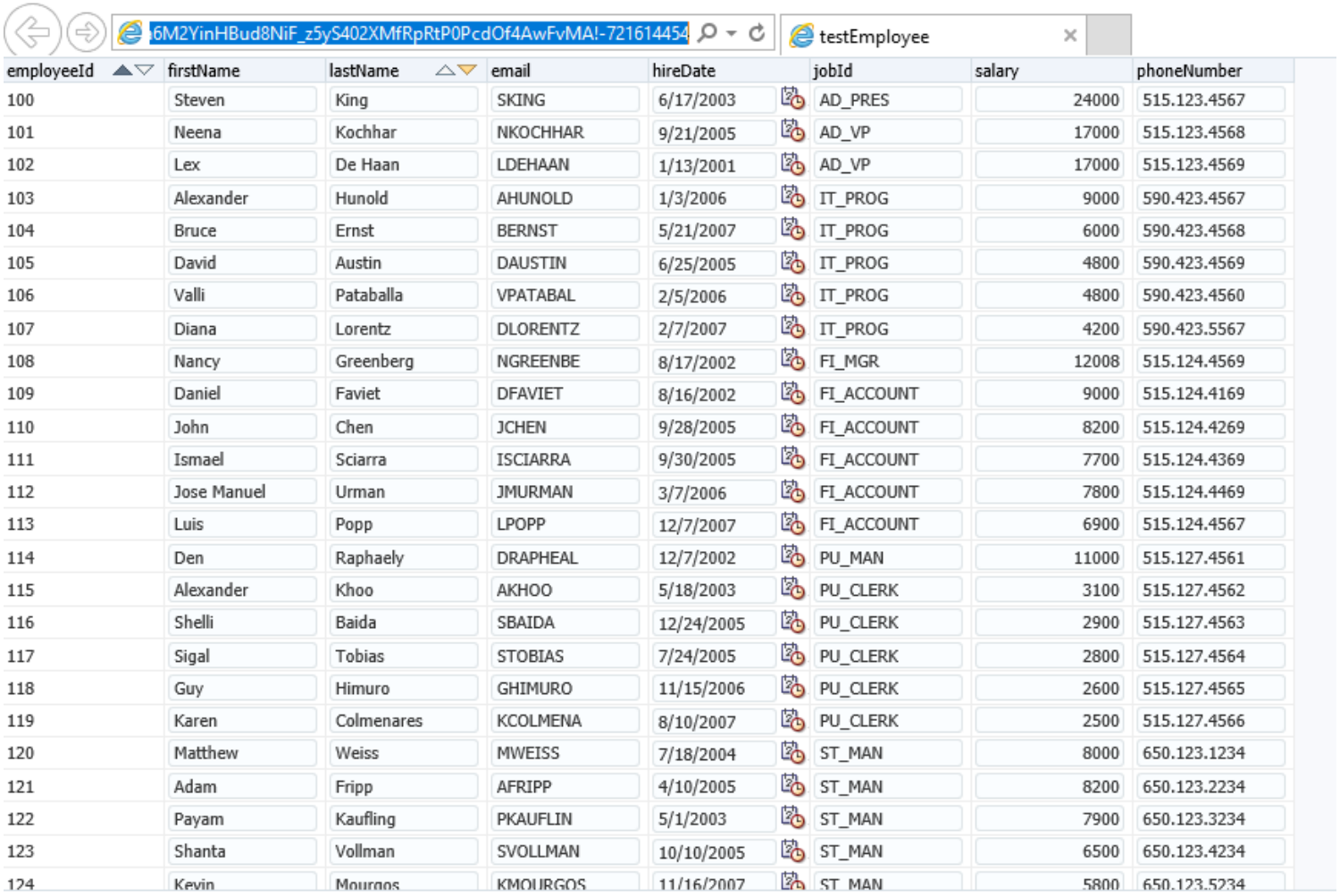
    Ex. int values[] = {109,100,104};

- Lists are sequence containers or collections.
  - List<T> and T is any type or class

    Ex. List List<Employee>

Objective: We want not just to store values but also to be able to quickly access each individual value.

Considering a List of Employees



employeeId	firstName	lastName	email	hireDate	jobId	salary	phoneNumber
100	Steven	King	SKING	6/17/2003	AD_PRES	24000	515.123.4567
101	Neena	Kochhar	NKOCHHAR	9/21/2005	AD_VP	17000	515.123.4568
102	Lex	De Haan	LDEHAAN	1/13/2001	AD_VP	17000	515.123.4569
103	Alexander	Hunold	AHUNOLD	1/3/2006	IT_PROG	9000	590.423.4567
104	Bruce	Ernst	BERNST	5/21/2007	IT_PROG	6000	590.423.4568
105	David	Austin	DAUSTIN	6/25/2005	IT_PROG	4800	590.423.4569
106	Valli	Pataballa	VPATABAL	2/5/2006	IT_PROG	4800	590.423.4560
107	Diana	Lorentz	DLORENTZ	2/7/2007	IT_PROG	4200	590.423.5567
108	Nancy	Greenberg	NGREENBE	8/17/2002	FI_MGR	12008	515.124.4569
109	Daniel	Faviet	DFAVIET	8/16/2002	FI_ACCOUNT	9000	515.124.4169
110	John	Chen	JCHEN	9/28/2005	FI_ACCOUNT	8200	515.124.4269
111	Ismael	Sciarra	ISCIARRA	9/30/2005	FI_ACCOUNT	7700	515.124.4369
112	Jose Manuel	Urman	JMURMAN	3/7/2006	FI_ACCOUNT	7800	515.124.4469
113	Luis	Popp	LPOPP	12/7/2007	FI_ACCOUNT	6900	515.124.4567
114	Den	Raphaely	DRAPHEAL	12/7/2002	PU_MAN	11000	515.127.4561
115	Alexander	Khoo	AKHOO	5/18/2003	PU_CLERK	3100	515.127.4562
116	Shelli	Baida	SBAIDA	12/24/2005	PU_CLERK	2900	515.127.4563
117	Sigal	Tobias	STOBIAS	7/24/2005	PU_CLERK	2800	515.127.4564
118	Guy	Himuro	GHIMURO	11/15/2006	PU_CLERK	2600	515.127.4565
119	Karen	Colmenares	KCOLMENA	8/10/2007	PU_CLERK	2500	515.127.4566
120	Matthew	Weiss	MWEISS	7/18/2004	ST_MAN	8000	650.123.1234
121	Adam	Fripp	AFRIPP	4/10/2005	ST_MAN	8200	650.123.2234
122	Payam	Kaufling	PKAUFLIN	5/1/2003	ST_MAN	7900	650.123.3234
123	Shanta	Vollman	SVOLLMAN	10/10/2005	ST_MAN	6500	650.123.4234
124	Kevin	Mourgos	KMOURGOS	11/16/2007	ST_MAN	5800	650.123.5234

Why is sorting data important?

How Sort Works for a List<Employee>

Declaration:

List<Employee> list = ArrayList<>();

// Call sort method

sort(list, new EmployeeIdComparator());

sort(list, new NameComparator());

sort(list, new JobIdComparator());

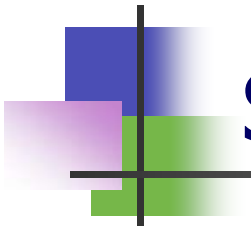
sort(list, new HireDateComparator());

Class Definition of Employee:

```
public class Employee {
    int employeeId;
    String name;
    String jobId;
    Date hireDate;
```

Example of sort method:

```
public static <T> void sort(List<T> list, Comparator<T> c) {
        Collections.sort(list, c);
}
```



Sorting Algorithms

Simple

- Bubble Sort
- Selection Sort
- Inserting Sort

Advanced

- Heap Sort
- Quick Sort
- Merge Sort
- Shell Sort
- …

Bubble Sort Algorithm

- Compare every two elements, swap the max to the end.
- Scan from start to end, swap the max to the end.
- Continuing scan next two elements, swap the max to the end, until the end of the array.
- The data is now arranged in ascending.

Programming Logic:

if a[j]>a[j+1], swap(a[j], a[j+1]);

How it works

```
67      // ********************************************************
68      // The bubbleSort function performs an ascending order    *
69      // bubble sort on the array. The size parameter is the    *
70      // number of elements in the array.                       *
71      // ********************************************************
72      public static void bubbleSort(int[] array) {
73          int swap;
74          for (int i = 0; i < (array.length - 1); i++) {
75              for (int j = 0; j < (array.length - i - 1); j++) {
76                  /* For descending order use < */
77                  if (array[j] > array[j + 1]) {
78                      swap = array[j];
79                      array[j] = array[j + 1];
80                      array[j + 1] = swap;
81                  }
82              }
83          }
84      }
```

Volgh:

MS4# Mlp#Skdp/#62;2534:

Selection Sort Algorithm

- First locate the smallest value in the array and move that value to the beginning of the array.
- From 1st to last, find the min value, store to the 1st position.
- From 2nd to last, find the min value, store to the 2nd position.
- ...
- This process continues until all the data is ordered.

Programming Logic:

```
index=i;  // store the current start position
if (a[j]<a[index]) {index=j;} // store the index of min value
```

How it works

```
86      // ********************************************************
87      // The selectionSort function performs an ascending order *
88      // bubble sort on the array. The size parameter is the    *
89      // number of elements in the array.                       *
90      // ********************************************************
91      public static void selectionSort(int[] array) {
92          for (int i = 0; i < array.length - 1; i++) {
93              int index = i;
94              for (int j = i + 1; j < array.length; j++) {
95                  if (array[j] < array[index])
96                      index = j;
97              }
98              int smallerNumber = array[index];
99              array[index] = array[i];
100             array[i] = smallerNumber;
101         }
102     }
```

Insertion Sort Algorithm

- Insertion sort algorithm sorts the list by moving each element to its proper place
- For each element a[i], the array a[0..i-1] is already sorted. Scan a[0..i-1], find the correct place and insert a[i].

Programming Logic:

- Find the correct place and insert the a[i] in sorted a[0..i-1].
- Keep moving each element to its right (a[j] = a[j-1]), until the next element is less than a[i].

How it works

```
110     // ***************************************************
111     // The InsertionSort function performs an ascending order *
112     // bubble sort on the array. The size parameter is the    *
113     // number of elements in the array.                       *
114     // ***************************************************
115     private static void insertionSort(int[] array) {
116         for (int i = 1; i < array.length; i++) {
117             int valueToSort = array[i];
118             int j = i;
119             while (j > 0 && array[j-1] > valueToSort) {
120                 array[j] = array[j-1];
121                 j--;
122             }
123             array[j] = valueToSort;
124         }
125     }
```

The Efficiency of Sorting Algorithms

- ***Bubble sort*** is simple sorting algorithm and is too slow and impractical (O (n^2)).
- ***Selection sort*** is simple sorting algorithm, specifically an in-place comparison sort. It is inefficient on large lists (O (n^2)).
- ***Insertion sort*** is a simple sorting algorithm that builds the final sorted array (or list) one item at a time. It is inefficient on large lists. (O (n^2)).
- ***Advanced sorting algorithms***:
  - Heap sort (O (n log n))
  - Quick sort (O (n log n))
  - Merge sort (O (n log n))

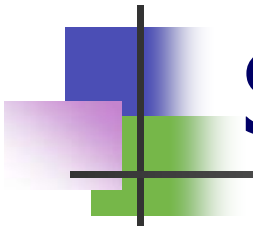
```
35  //*********************************************
36  // quickSort uses the quicksort algorithm to     *
37  // sort set, from set[start] through set[end].   *
38  //*********************************************
39  void quickSort(int set[], int start, int end)
40  {
41      int pivotPoint;
42
43      if (start < end) {
44          // Get the pivot point.
45          pivotPoint = partition(set, start, end);
46          // Sort the first sub list.
47          quickSort(set, start, pivotPoint - 1);
48          // Sort the second sub list.
49          quickSort(set, pivotPoint + 1, end);
50      }
51  }
```

Example how sort works using OO Programming

```
20      public static void main(String[] args) throws ParseException {
21          List<Employee> list = Arrays.asList(
22              new Employee(109, "Joe", "Support", sdf.parse("03/06/2017")),
23              new Employee(100, "Pete", "Engineer", sdf.parse("02/13/2017")),
24              new Employee(104, "Chris", "Manager", sdf.parse("03/09/2017")));
25          // Sort by employeeId
26          sort(list, new EmployeeIdComparator());
27          // Sort by name
28          sort(list, new NameComparator());
29          // Sort by hireDate
30          sort(list, new HireDateComparator());
31      }
```

Result:

```
C:\Oracle\Middleware\Oracle_Home\oracle_common\jdk\bin\javaw.exe -server -classpath C:\Users\jpham\Desktop\JDeveloper\MyWork\SortingDemo
[{Id=100, Name=Pete, HireDate: 2/13/2017}, {Id=104, Name=Chris, HireDate: 3/09/2017}, {Id=109, Name=Joe, HireDate: 3/06/2017}]
[{Id=104, Name=Chris, HireDate: 3/09/2017}, {Id=109, Name=Joe, HireDate: 3/06/2017}, {Id=100, Name=Pete, HireDate: 2/13/2017}]
[{Id=100, Name=Pete, HireDate: 2/13/2017}, {Id=109, Name=Joe, HireDate: 3/06/2017}, {Id=104, Name=Chris, HireDate: 3/09/2017}]
```



Summary

- ***Lists***
  - Sorting data is important and necessary.
- ***Bubble Sort***
  - a straightforward and simplistic method of sorting data.
- ***Selection Sort***
  - an in-place comparison sort.
- ***Insertion Sort***
  - a simple sorting algorithm that is relatively efficient for small lists and mostly-sorted lists.
- ***Advanced Sorting Algorithms***
  - Heap Sort, Quick Sort, Merge Sort, …
- ***How Sort Works using OO Programming***

Questions